

제 2장 윈도우 기본 입출력

2026년 1학기 윈도우 프로그래밍

2장 학습 목표

- 학습목표

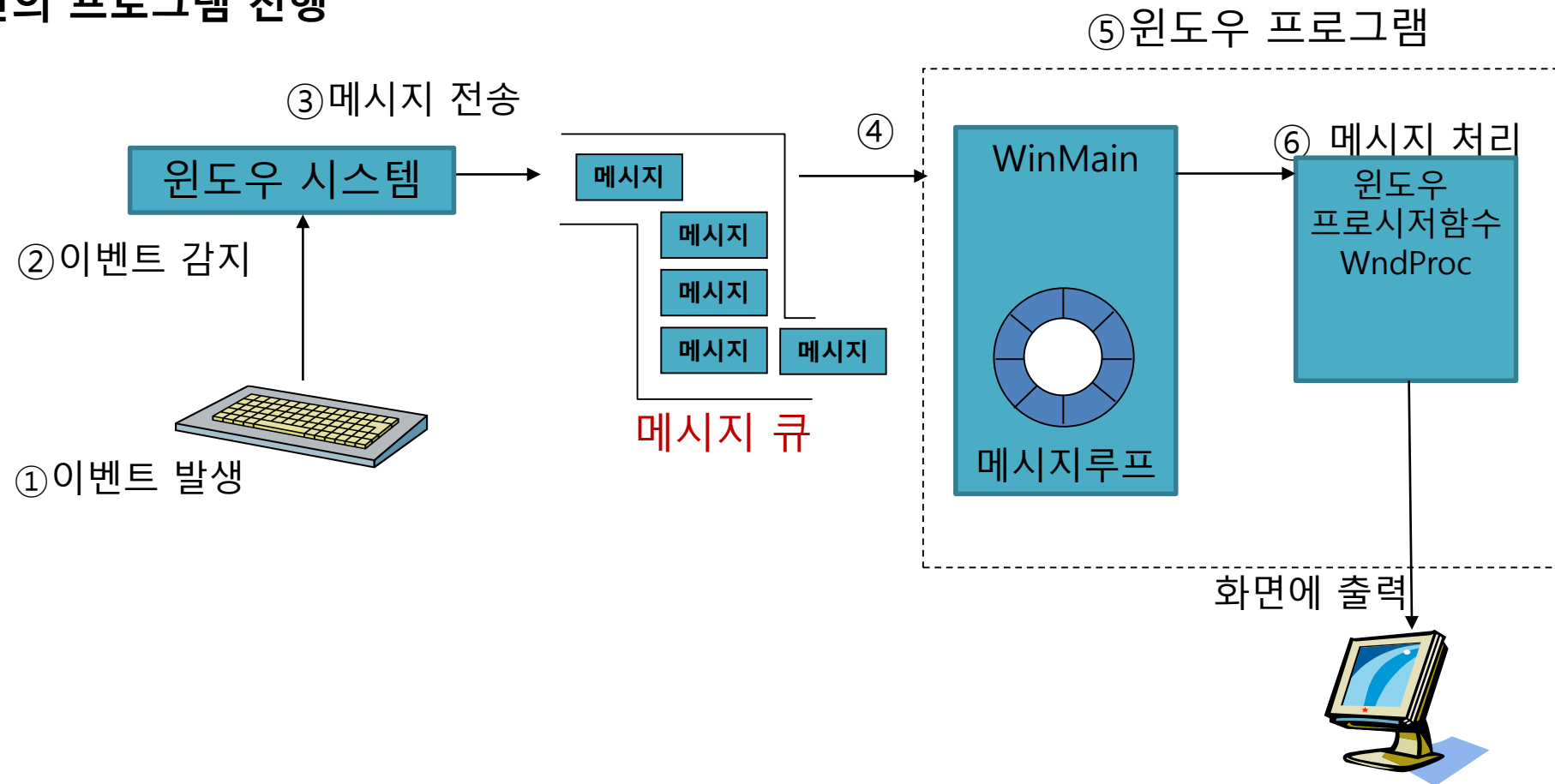
- 윈도우 화면에 출력하기 위해 디바이스 컨텍스트 개념을 이해할 수 있다.
- 텍스트를 출력하는 기본 함수를 사용할 수 있다.
- 기본 도형을 화면에 출력할 때 필요한 요소와 함수를 사용할 수 있다.

- 내용

- 출력 영역 얻기
- 텍스트 출력하기
- 키보드 메시지 처리하기
- Caret 이용하기
- 직선, 원, 사각형, 다각형 그리기

윈도우 프로그램 특징

- 메시지 기반의 프로그램 진행



- 윈도우 프로그램은 윈도우에서 발생하는 이벤트에 의한 메시지를 처리하며 프로그램이 진행된다.

윈도우 메시지들

- 마우스/키보드 메시지, 윈도우 메시지, 시스템 메시지 등 수백 개의 메시지가 발생되어 처리됨

```
#include <windows.h>          //--- 윈도우 헤더 파일
#include <tchar.h>

HINSTANCE g_hInst;
LPCTSTR lpszClass = L"Window_Class Name";
LPCTSTR lpszWindowName = L"Window Programming Lab";

LRESULT CALLBACK WndProc (HWND hWnd, UINT iMessage, WPARAM wParam, LPARAM lParam);

int WINAPI WinMain (HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR lpszCmdParam, int nCmdShow)
{
    HWND hWnd;
    MSG Message;
    WNDCLASSEX WndClass;
    g_hInst = hInstance;

    WndClass.cbSize = sizeof(WndClass);
    WndClass.style = CS_HREDRAW | CS_VREDRAW;
    WndClass.lpfnWndProc = (WNDPROC)WndProc;
    WndClass.cbClsExtra = 0;
    WndClass.cbWndExtra = 0;
    WndClass.hInstance = hInstance;
    WndClass.hIcon = LoadIcon(NULL, IDI_APPLICATION);
    WndClass.hCursor = LoadCursor(NULL, IDC_ARROW);
    WndClass.hbrBackground = (HBRUSH)GetStockObject(BLACK_BRUSH);
    WndClass.lpszMenuName = NULL;
    WndClass.lpszClassName = lpszClass;
    WndClass.hIconSm = LoadIcon(NULL, IDI_APPLICATION);
    RegisterClassEx(&WndClass);

    hWnd = CreateWindow (lpszClass, lpszWindowName, WS_OVERLAPPEDWINDOW, 0, 0, 800, 600, NULL, (HMENU)NULL, hInstance, NULL);
    ShowWindow (hWnd, nCmdShow);
    UpdateWindow (hWnd);

    while (GetMessage (&Message, 0, 0, 0)) {
        TranslateMessage (&Message);
        DispatchMessage (&Message);
    }
    return Message.wParam;
}
```

```
LRESULT CALLBACK WndProc (HWND hWnd, UINT uMsg, WPARAM wParam, LPARAM lParam)
{
    PAINTSTRUCT ps;
    HDC hDC;

    //--- 메시지 처리하기
    switch (uMsg) {
        case WM_CREATE:
            break;
        case WM_PAINT:
            hDC = BeginPaint(hWnd, &ps);
            EndPaint(hWnd, &ps);
            break;
        case WM_DESTROY:
            PostQuitMessage(0);
            break;
    }
    return DefWindowProc (hWnd, uMsg, wParam, lParam);
}
```

- 메인 함수

int WINAPI WinMain (HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR lpszCmdParam, int nCmdShow)

- hInstance: 현재 실행중인 어플리케이션의 인스턴스 핸들
- hPrevInstance: 동일한 어플리케이션이 실행중일 경우 이전에 실행된 프로그램의 인스턴스 핸들.
- lpszCmdLine: 커멘드라인 상에서 프로그램 구동 시 전달된 문자열
- nCmdShow: 윈도우가 화면에 출력될 형태

- WinMain 함수에 대한 경고:

- VS 코드 분석 도구가 코드 오류를 찾기위해 사용하는 주석이 없어서 발생
- 해소하려면:
- int WINAPI WinMain (_In_ HINSTANCE hInstance, _In_opt_ HINSTANCE hPrevInstance, _In_ LPSTR lpszCmdParam, _In_ int nCmdShow) 로 선언
- SAL (Source-code Annotation Language) 도움말:
<https://learn.microsoft.com/ko-kr/previous-versions/visualstudio/visual-studio-2015/code-quality/understanding-sal?view=vs-2015&redirectedfrom=MSDN>

- 메시지 처리 함수

LRESULT CALLBACK WndProc (HWND hWnd, UINT iMsg, WPARAM wParam, LPARAM lParam)

- hWnd: 메시지가 발생한 윈도우 핸들
- iMsg: 발생한 메시지 (위의 메시지들이 전달된다)**
- wParam, lParam: 메시지를 처리하기 위해 필요한 데이터들, 메시지들마다 다른 값이 전달된다.

윈도우 메시지들

- 마우스/키보드 메시지, 윈도우 메시지, 시스템 메시지 등 수백 개의 메시지가 발생되어 처리됨

WM_ACTIVATE
WM_ACTIVATEAPP
WM_APP
WM_APPCOMMAND
WM_CHAR
WM_CHARTOITEM
WM_CHILDACTIVATE
WM_CLEAR
WM_CLOSE
WM_COMMAND
WM_COMPACTING
WM_COMPAREITEM
WM_COPY
WM_CREATE
WM_CTLCOLORBTN
WM_CTLCOLORDLG
WM_CTLCOLOREDIT
WM_CTLCOLORLISTBOX
WM_CTLCOLORSCROLLBAR
WM_CTLCOLORSTATIC
WM_CUT
WM_DEADCHAR
WM_DELETEITEM
WM_DESTROY
WM_DRAWITEM
WM_DROPFILES
WM_ENABLE
WM_ENDSESSION
WM_ENTERIDLE
WM_ERASEBKGD
WM_FONTCHANGE
WM_GETDLGCODE
WM_GETFONT
WM_GETMINMAXINFO
WM_GETTEXT
WM_GETTEXTLENGTH
WM_HSCROLL

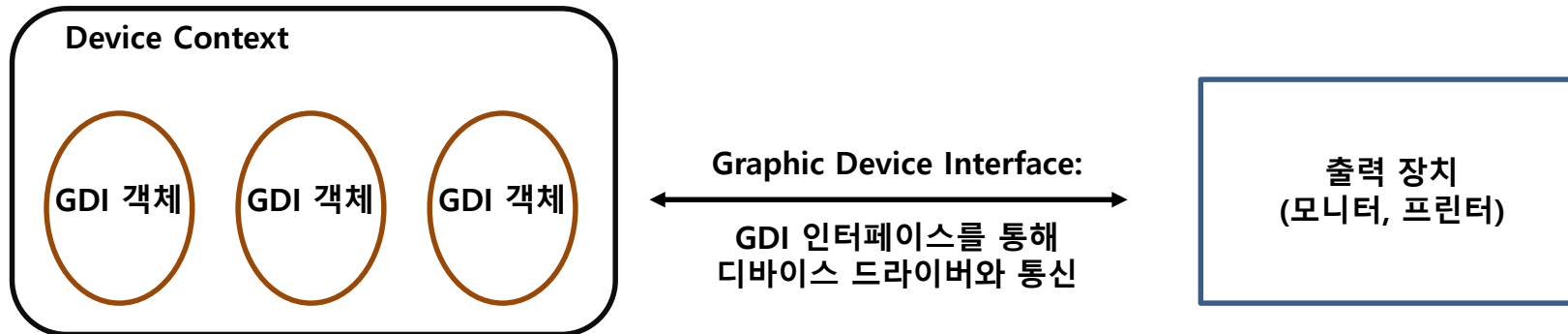
WM_ICONERASEBKGD
WM_INITDIALOG
WM_INITMENU
WM_INITMENUPOPUP
WM_KEYDOWN
WM_KEYUP
WM_KILLFOCUS
WM_LBUTTONDBLCLK
WM_LBUTTONDOWN
WM_LBUTTONUP
WM_MENUCHAR
WM_MENUSELECT
WM_MOUSEACTIVATE
WM_MOUSEMOVE
WM_MOVE
WM_NCACTIVATE
WM_NCCALCSIZE
WM_NCCREATE
WM_NCDESTROY
WM_NCHITTEST
WM_NEXTDLGCTL
WM_NULL
WM_OTHERWINDOWCREATED
WM_OTHERWINDOWDESTROYED
WM_PAINT
WM_PAINTCLIPBOARD
WM_PAINTICON
WM_PALETTECHANGED
WM_PARENTNOTIFY
WM_PASTE
WM_QUERYDRAGICON
WM_QUERYENDSESSION

WM_QUERYNEWPALETTE
WM_QUERYOPEN
WM_QUIT
WM_RBUTTONDOWN
WM_RBUTTONUP
WM_RENDERFORMAT
WM_SETCURSOR
WM_SETFOCUS
WM_SETFONT
WM_SERREDRAW
WM_SETTEXT
WM_SHOWWINDOW
WM_SIZE
WM_SIZECLIPBOARD
WM_SYSCHAR
WM_SYSCOLORCHANGE
WM_SYSCOMMAND
WM_SYSDEADCHAR
WM_SYSKEYDOWN
WM_SYSKEYUP
WM_TIMECHANGE
WM_TIMER
WM_UNDO
WM_USER
WM_VKEYTOITEM
WM_VSCROLL
WM_WININICHANGE

1. 출력 영역 얻기

- **GDI (Graphic Device Interface)**

- 윈도우에서 화면이나 프린터에 출력할 때 사용하는 API의 집합
- 윈도우 프로그램에서의 모든 출력은 GDI를 통해서 화면과 프린터로 나가게 되어 있다.
- GDI 객체: 그래픽 출력에 사용되는 도구
 - GDI 객체 종류: 펜, 브러시, 비트맵, 폰트 등
 - GDI 객체들은 핸들을 이용해서 사용한다.
- DC (Device Context): GDI 오브젝트를 모아놓은 것
- GDI는 현재 DC에 선택되어 있는 GDI 오브젝트를 사용한다.



1. 출력 영역 얻기

- **GDI+**

- C++ 기반으로 클래스의 집합체
 - 모든 그리기 함수들은 Graphics 클래스의 멤버 함수로 정의
 - 출력 함수들은 그리기에 필요한 값들을 인수로 전달받아 사용한다.
 - C++ 의 함수오버로딩 기능을 사용하고 있어서 여러 종류 인수 타입이 지원된다.
 - 함수가 종료될 때 자동으로 파괴자가 호출되므로 객체 해제를 직접 작성할 필요가 없다.
 - 동적 할당한 경우에는 삭제해야한다.
 - 모든 문자열은 유니코드여야 한다.
 - GDI에 비해 많은 기능이 추가로 제공된다.
- 여러 다양한 기능이 추가되고 사용도 편리하지만 GDI에 비해 속도가 느리다.

1. 출력 영역 얻기

구분	GDI (Graphics Device Interface)	GDI+
출시 시점	초기 Windows 버전부터 존재 (Win16/Win32)	Windows XP에서 도입
언어 기반	C API 중심	C++ 클래스 기반 객체 모델
좌표 처리	정수 좌표만 지원	부동소수점 좌표 지원 (더 정밀한 그래픽)
렌더링 품질	기본 2D 렌더링, 앤티앨리어싱 없음	앤티앨리어싱 지원 → 매끄러운 선과 곡선
색상/브러시	단순 색상, 제한된 패턴	그라데이션, 반투명, 텍스처 브러시 지원
이미지 포맷	BMP 중심, 제한적	JPEG, PNG, GIF 등 현대적 포맷 지원
변환 기능	제한적	Affine 변환(회전, 스케일, 이동 등) 지원
성능	빠르고 가볍지만 기능 제한	기능 풍부하지만 상대적으로 느림
상태	오래된 API, 유지보수 중심	XP 이후 권장되었지만 현재는 Direct2D로 대체 추세

이번 학기는 GDI를 기본으로 사용하기로 한다!

디바이스 컨텍스트: 출력 영역

- **DC (Device Context)**

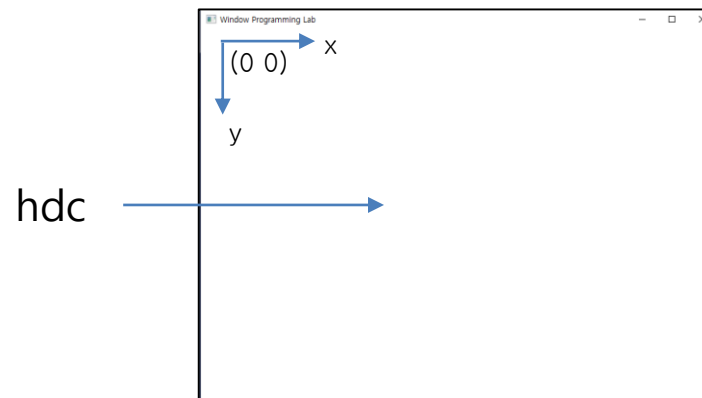
- 디스플레이 또는 프린터와 같은 출력에 필요한 정보를 포함하는 윈도우 데이터 구조체
 - 그래픽 관련한 선택 정보 (폰트, 색상, 굵기, 무늬, 출력 방법 등)를 모아 놓은 구조체
- 모든 그리기는 디바이스 컨텍스트 개체를 통해 수행된다.
 - 디바이스 컨텍스트는 GDI 모듈에 의해서 관리된다.
 - 간단한 출력은 디폴트로 설정된 속성 (예를 들어 선을 그린다면 선의 색상, 굵기, 모양 등) 이용
 - 속성은 얻어온 DC에서 변경 가능: 선의 형태, 색깔, 굵기, 모드, 위치 등의 속성 변경 가능

- 윈도우에서 출력 장치에 무언가 출력하기 위해서는 반드시 DC가 필요, DC 핸들을 얻은 후 해당 DC에 데이터를 출력한다.

- 윈도우의 화면 메모리에 그리고 그것들을 윈도우 운영체제에서 출력시켜준다.
- 이러한 화면 메모리를 제어하는 것이 DC
- **모든 그래픽 출력에 있어서 각각의 윈도우는 모두 DC 핸들(HDC)을 얻어야 한다.**
- DC 핸들은 출력대상을 나타내는 구분 번호로 생각
- 모든 GDI 함수들은 첫 번째 인자로 DC 핸들을 필요로 한다.

- DC의 유형

- 화면 출력을 위한 디스플레이 DC
- 프린터나 플로터 출력을 위한 프린터 DC
- 비트맵 출력을 위한 메모리 DC
- 디바이스 정보를 얻기 위한 정보 DC



- 디바이스 컨텍스트 얻어오는 방법
 - 다양한 함수 호출을 통하여 디바이스 컨텍스트를 얻어온다.

DC 얻어오는 함수	DC 해제하는 함수	기능
BeginPaint()	EndPaint():	WM_PAINT메시지에서 DC를 얻거나 반환할 때 사용
GetDC()	ReleaseDC():	WM_PAINT 메시지 외의 일반 메시지에서 DC를 얻거나 반환할 때 사용
CreateDC()	DeleteDC():	DC를 만들어 사용
CreateIC()	DeleteDC():	DC에 출력하지 않고 정보만 얻고자 할 때 사용
CreateCompatibleDC()	DeleteDC():	이미 있는 DC와 같은 또 하나의 DC만들 때 사용. 보통 디스플레이를 이용한 메모리 DC를 만들 때 사용

디바이스 컨텍스트 얻어오기

- 일반 메시지에서 DC를 얻어오는 GetDC() 함수 / 해제하는 ReleaseDC () 함수

HDC GetDC (HWND hWnd);

- HWND hWnd: 생성된 윈도우의 핸들값
- 리턴 값으로 디바이스 컨텍스트 핸들을 얻어온다.

int ReleaseDC (HWND hWnd, HDC hDC);

- HWND hWnd: 해제할 DC에 대한 윈도우 핸들
- HDC hDC: 해제할 DC 핸들

디바이스 컨텍스트 얻어오기

- WM_PAINT 메시지에서 DC를 얻어오는 BeginPaint() 함수 / 해제하는 EndPaint () 함수

HDC BeginPaint (HWND hWnd, PAINTSTRUCT *lpPaint);

- HWND hWnd: 생성된 윈도우의 핸들값
- PAINTSTRUCT *lpPaint: 출력될 영역에 대한 정보를 저장한 구조체 공간에 대한 주소
- 리턴 값으로 디바이스 컨텍스트 핸들을 얻어온다

BOOL EndPaint (HWND hWnd, PAINTSTRUCT *lpPaint);

- HWND hWnd: 생성된 윈도우의 핸들값
- PAINTSTRUCT *lpPaint: 출력될 영역에 대한 정보를 저장한 구조체 공간에 대한 주소

- PAINTSTRUCT 구조체

```
typedef struct tagPAINTSTRUCT {  
    HDC hdc;                //---그리고자 하는 DC 핸들  
    BOOL fErase;            //--- 배경 삭제 여부를 가리킨다: 0이 아니면 다시 그린다.  
    RECT rcPaint;           //--- 다시 그릴 사각형의 좌표값  
    BOOL fRestore;  
    BOOL fIncUpdate;  
    BYTE rgbReserved[16];  
} PAINTSTRUCT;
```

디바이스 컨텍스트 얻어오기

- 사용 예)

```
int WINAPI WinMain (HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR lpszCmdParam, int nCmdShow)
{
    ...
}
```

```
LRESULT CALLBACK WndProc (HWND hWnd, UINT iMsg, WPARAM wParam, LPARAM lParam)
{
    HDC hDC;
    PAINTSTRUCT ps;

    switch (iMsg)
    {
        case WM_PAINT:
            hDC = BeginPaint (hWnd, &ps);
            EndPaint (hWnd, &ps);
            break;

        case WM_TIMER:
            hDC = GetDC (hWnd);
            ReleaseDC (hWnd, hDC);
            break;

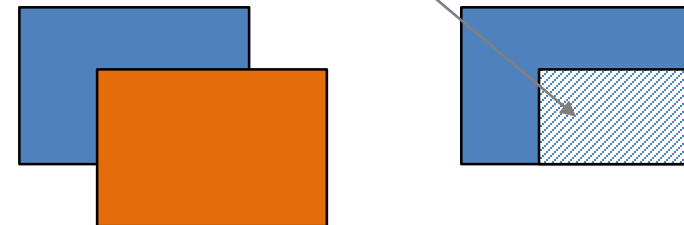
        case WM_DESTROY:
            PostQuitMessage ( 0 );
            break;
    }
    return DefWindowProc (hWnd, iMsg, wParam, lParam);
}
```

//--- WM_PAINT 메시지: dc를 얻어 그리기 진행
//--- BeginPaint 함수를 사용하여 DC를 얻는다

//--- WM_TIMER 메시지: dc를 얻어 출력 진행
//--- GetDC 함수를 사용하여 DC를 얻는다

2. 메시지 처리: WM_PAINT 메시지

- 클라이언트 영역에 그릴 필요가 있을 때: **WM_PAINT** 메시지 사용
 - 원도우의 크기가 변경되었을 때나 다른 윈도우에 가려져 있다가 드러날 때 등 화면에 출력된 결과가 깨질 수 있다.
 - 화면의 그래픽 일부가 깨지거나 다시 출력해야 할 필요가 있는 영역을 **무효화(invalid) 영역**이라 하고, 이러한 경우를 화면이 무효화 되었다고 한다.
 - OS는 깨진 화면을 복구해주지 않는다.
 - 단지 OS는 화면이 깨질 때 마다 **WM_PAINT** 메시지를 발생시켜준다.
 - 즉, 윈도우의 클라이언트 영역 중 일부가 무효화(invalid)되면 OS가 WM_PAINT 메시지를 큐에 넣어준다.
 - 그래서 **출력은 WM_PAINT 메시지 아래에서 해야 한다!!**
 - 그래야 화면이 깨질 때 마다 WM_PAINT 메시지가 발생하고, 그 아래에 작성한 소스가 다시 실행되어 화면이 복구된다!
- 다음과 같은 경우에 OS는 WM_PAINT 메시지를 프로그램에 전달한다.
 - 원도우가 처음 생성되었을 때
 - 원도우의 위치가 이동되었을 때
 - 원도우의 크기가 변경되었을 때
 - 최대, 최소화되었을 때
 - 다른 윈도우에 가려져 있다가 드러날 때
 - 파일로부터 데이터를 출력할 때
 - 출력된 데이터의 일부분을 스크롤, 선택, 변화시킬 때
 - InvalidateRect()**, 또는 **InvalidateRgn()** 함수를 호출하여 강제로 화면을 무효화시킬 때



메시지 처리: WM_PAINT 메시지

- WM_PAINT 메시지는
 - 이 메시지를 받았을 때 프로그램은 화면 복구를 위해 클라이언트 영역 전체 또는 무효화된 부분만 다시 그려야 한다.
 - OS는 화면이 무효화될 때 클라이언트 영역을 복구해 주지 않는 대신에 이 메시지를 보내 줌으로써 해당 프로그램에게 다시 그려야 할 시점을 알려 준다.
 - 따라서, 클라이언트 영역에 출력한 정보는 모두 저장해 두어야 복구가 가능하다.
 - WM_PAINT 메시지는 모든 메시지 중에서 우선 순위가 가장 낮다.
 - GetMessage() 함수는 메시지 큐에 WM_PAINT 메시지가 있더라도 다른 메시지가 대기 중이면 그 메시지를 먼저 처리한다.
 - WM_PAINT 메시지는 큐에 대기 중인 다른 메시지가 없고 무효화 영역이 존재할 때만 윈도우 프로시저로 보내진다.
 - WM_PAINT 메시지는 한번에 하나만 메시지 큐에 들어갈 수 있다.
 - 만약 무효화 영역이 생겼는데 WM_PAINT 메시지가 이미 메시지 큐에 있으면 기존의 무효화 영역과 새 무효화 영역의 합으로 새로운 무효화 영역이 설정된다.
- 해당 윈도우 프로시저에서 이 메시지 (WM_PAINT)를 처리하지 않으면 이 메시지는 DefWindowProc() 함수가 처리한다.
 - 이 함수는 무효 영역을 모두 유효화(valid)하며 다시 그리기는 하지 않는다.
- WM_PAINT 메시지에서 그리기를 할 때는 BeginPaint()와 EndPaint() 함수를 사용해야 한다.
 - 이 두 함수는 WM_PAINT 메시지 내에서만 사용된다.
 - 다시 그려야 할 영역에 대한 정확한 좌표를 조사하며 무효 영역을 유효화하고 캐럿을 숨기거나 배경을 지우는 등의 꼭 필요한 동작을 한다.
- 다른 메시지에서도 출력은 가능하다!

3. 메시지 처리: 윈도우 기본 메시지

- 윈도우가 생성될 때: **WM_CREATE** 메시지
 - CreateWindow 함수에 의해 윈도우가 생성될 때 호출되는 메시지
 - 메모리에 윈도우를 생성한 후 화면에 보이기 전에 보내지며 주로 윈도우에 관련된 초기화 작업을 할 때 사용됨
 - 윈도우 동작을 위한 메모리 할당, 리소스 생성, 차일드 컨트롤 생성, 윈도우 속성 초기화 작업에 이 메시지가 사용된다.
 - CreateWindow 함수는 이 메시지를 완전히 처리한 후에 리턴
 - wParam: 0
 - lParam: CREATESTRUCT * 구조체, 즉 윈도우가 생성될 때 필요한 초기화 정보

```
typedef struct tagCREATESTRUCT {  
    LPVOID lpCreateParams;           // CreateWindowEx에서 전달한 사용자 데이터  
    HINSTANCE hInstance;             // 인스턴스 핸들  
    HMENU hMenu;                     // 메뉴 핸들  
    HWND hwndParent;                // 부모 윈도우 핸들  
    int cy;                           // 윈도우 높이  
    int cx;                           // 윈도우 너비  
    int y;                            // 윈도우 Y 좌표  
    int x;                            // 윈도우 X 좌표  
    LONG style;                       // 윈도우 스타일  
    LPCSTR lpszName;                 // 윈도우 이름(제목)  
    LPCSTR lpszClass;                // 클래스 이름  
    DWORD dwExStyle;                 // 확장 스타일  
} CREATESTRUCT;
```


3. 메시지 처리: 윈도우 기본 메시지

- 윈도우를 닫을 때: **WM_CLOSE** 메시지
 - 사용자가 닫기 버튼을 누르면 발생하는 메시지
 - wParam, lParam: 사용되지 않음 (항상 0)
- 윈도우를 파괴할 때: **WM_DESTROY** 메시지
 - 해당 윈도우에 대해 DestroyWindow가 실제로 수행될 때 발생하는 메시지. 그 윈도우의 윈도우 프로시저로 메시지 전달
 - 파일 닫기, 메모리 해제같은 정리 작업 수행. 메인 윈도우라면 PostQuitMessage 호출해서 어플리케이션을 종료
 - wParam, lParam: 사용되지 않음 (항상 0)
- 프로그램을 종료할 때: **WM_QUIT** 메시지
 - 프로그램을 종료하고 윈도우를 파괴할 때 호출되는 메시지
 - **PostQuitMessage** 함수를 호출하면 발생한다. 즉, WM_QUIT은 사용자가 직접 발생시키는 메시지가 아니라, 프로그램이 PostQuitMessage를 호출해서 발생시킨다.
 - WM_QUIT 메시지를 받으면 메시지 루프의 GetMessage 함수 리턴값이 False가 되어 프로그램이 종료
 - 대부분의 경우 WM_DESTROY 메시지에서 호출하는 것이 올바른 종료 흐름이고, 직접 호출은 예외적인 상황에서만 사용
 - wParam: PostQuitMessage에 전달한 종료 코드
 - lParam: 사용되지 않음 (항상 0)
- 종료 메시지를 만들기 위하여 **PostQuitMessage ()** 함수를 호출하여 프로그램을 종료한다.
 - 파괴되는 윈도우가 메인 윈도우일 경우에는 반드시 PostQuitMessage 함수를 호출하여 메시지 루프를 종료하도록 한다.

3. 메시지 처리: 윈도우 기본 메시지

- 프로그램 종료 함수

VOID PostQuitMessage (int nExitCode);

- 스레드 메시지 큐에 WM_QUIT 메시지를 붙이고 즉시 리턴
- WM_QUIT 메시지를 큐에 붙임으로써 시스템에게 이 스레드가 종료될 것이라는 것을 미리 알려준다.
- 메시지 루프는 보통 WM_QUIT 메시지를 받으면 종료하도록 되어 있으므로 이 함수를 호출하면 메시지 루프가 종료된다.

BOOL DestroyWindow (HWND hWnd);

- hWnd 윈도우를 파괴한다. 단, 이 함수로 다른 스레드에서 생성한 윈도우를 파괴할 수는 없다

메시지 처리: 윈도우 기본 메시지

- 사용 예)

```
LRESULT CALLBACK WndProc (HWND hWnd, UINT iMsg, WPARAM wParam, LPARAM lParam)
```

```
{
```

```
    HDC hDC;
```

```
    PAINTSTRUCT ps;
```

```
    switch (iMsg)
```

```
    {
```

```
        case WM_CREATE:
```

```
        break;
```

```
//--- WM_CREATE 메시지: 윈도우 생성될 때 호출, 필요한 초기화 작업
```

```
        case WM_PAINT:
```

```
            hDC = BeginPaint (hWnd, &ps) ;
```

```
//--- WM_PAINT 메시지: dc를 얻어 그리기 진행
```

```
//--- 필요한 그리기를 실행한다.
```

```
            EndPaint (hWnd, &ps) ;
```

```
        break;
```

```
        case WM_DESTROY:
```

```
            PostQuitMessage ( 0 );
```

```
//--- 프로그램 종료
```

```
        break;
```

```
    }
```

```
    return DefWindowProc (hWnd, iMsg, wParam, lParam);
```

```
}
```

메시지 처리: 윈도우 기본 메시지

- 윈도우 크기가 변경될 때: **WM_SIZE** 메시지

- wParam: 메시지가 발생한 이유
 - SIZE_MAXHIDE: 다른 윈도우가 최대화 되어 이 윈도우가 가려졌음
 - SIZE_MAXIMIZED: 최대화
 - SIZE_MAXSHOW: 다른 윈도우가 원래 크기로 복구되어 이 윈도우가 나타났음
 - SIZE_MINIMIZED: 최소화
 - SIZE_RESTORED: 크기가 변경
- lParam: 윈도우 높이와 폭 저장
 - HIWORD(lParam): 윈도우의 높이
 - LOWORD(lParam): 윈도우의 폭

• **HIWORD()**: 32bit 데이터에서 상위 16bit 데이터를 구하기 위한 매크로 함수
• **LOWORD()**: 32bit 데이터에서 하위 16bit 데이터를 구하기 위한 매크로 함수

- 윈도우 위치가 변경될 때: **WM_MOVE** 메시지

- wParam: 사용되지 않음
- lParam: 변경된 윈도우의 x와 y의 좌표값
 - HIWORD(lParam): 윈도우의 새로운 y 좌표
 - LOWORD(lParam): 윈도우의 새로운 x 좌표

4. 문자 집합

구분	char	TCHAR	WCHAR
기본 의미	8비트 문자 타입 (C 표준 char)로 ANSI/멀티바이트 문자 표현에 사용.	빌드 설정에 따라 char 또는 wchar_t로 바뀌는 제네릭 문자 타입.	16비트 와이드 문자 타입 (wchar_t에 대한 Windows 별칭)로 UTF-16 유니코드 문자 표현.
크기	보통 1바이트 (코드 페이지에 따라 문자 1개 또는 일부만 표현).	ANSI 빌드: 1바이트, 유니코드 빌드: 2바이트.	보통 2바이트 (UTF-16 코드 유닛 1개).
주 용도	레거시 ANSI 문자열, 코드 페이지 기반 멀티바이트 문자열 처리,	같은 소스를 ANSI/유니코드 둘 다 빌드할 때 사용,	유니코드(UTF-16) 기반의 최신 Windows API,
전형적인 문자열 타입	char*, LPSTR, LPCSTR.	LPTSTR, LPCTSTR (실제로는 LPSTR/LPWSTR로 치환).	WCHAR*, LPWSTR, LPCWSTR.
문자열 리터럴 작성	"text" 그대로 사용, 한글·유니코드는 코드 페이지 의존.	<u>_</u> T("text") 또는 TEXT("text") 사용, ANSI 빌드에서는 "text", 유니코드 빌드에서는 L"text"로 확장.	L"text" 와이드 문자열 리터럴 사용.
대표 API / 함수	MessageBoxA, CreateFileA, strcpy, strlen 등 A/표준 함수와 함께 사용.	MessageBox, CreateWindow, _tcscopy, _tprintf 등 T계열 매크로와 함께 사용 (내부적으로 A 또는 W로 연결).	MessageBoxW, CreateFileW, wcscopy, wcslen 등 W/와이드 함수와 함께 사용.
간단 예제	char sz[] = "Hi"; MessageBoxA(NULL, sz, "Title", 0);	#include <TCHAR.H> TCHAR sz[] = TEXT("Hi"); MessageBox(NULL, sz, TEXT("Title"), 0);	WCHAR sz[] = L"Hi"; MessageBoxW(NULL, sz, L"Title", 0);

문자열 자료형

- 문자열 자료형

API 자료형	같은 의미의 자료형	설명
LPSTR	char *	ANSI 코드 문자열 포인터
LPCSTR	const char *	ANSI 코드 문자열 포인터 상수
LPTSTR	TCHAR *	TCHAR 문자열 포인터
LPCTSTR	const TCHAR *	TCHAR 문자열 포인터 상수
LPWSTR	WCHAR *	유니코드 문자열 포인터
LPCWSTR	const WCHAR *	유니코드 문자열 포인터 상수

- LP : long pointer
- C : const
- T : TCHAR (유니코드, 멀티바이트 모두 지원)
- W: WCHAR (유니코드 문자열)
- STR : 문자열

5. 텍스트 출력하기

- 한 점 기준 텍스트 출력 함수

BOOL TextOut (HDC hDC, int x, int y, LPCTSTR lpString, int nLength);

- HDC hDC: BeginPaint()나 GetDC()를 통해 얻어온 DC핸들
- int x, int y: 텍스트를 출력할 좌표의 x값과 y값
- LPCTSTR lpString: 출력할 텍스트
- int nLength: 출력할 텍스트의 길이

- 문자열 관련 함수들:

내용	멀티바이트	TCHAR	WCHAR	API 문자열	사용 예
문자열 복사	strcpy	_tcscopy	wscopy	lstrcpy	strcpy(a, b)
문자열을 c 만큼 복사	strncpy	_tcsncpy	wcsncpy	lstrcpyn	strncpy (a, b, c)
문자열 길이	strlen	_tcslen	wcslen	lstrlen	strlen(a)
문자열 붙이기	strcat	_tcscat	wcscat	lstrcat	strcat (a, b)

윈도우에 “Hello World” 출력하기

- 윈도우 프로시저 함수 안에서 WM_PAINT 메시지에서 출력 처리

```
LRESULT CALLBACK WndProc (HWND hWnd, UINT iMsg, WPARAM wParam, LPARAM lParam)
{
    HDC hDC;
    PAINTSTRUCT ps;
    WCHAR str[100] = L"Second Hello World";
    int num;

    switch (iMsg)
    {
        case WM_PAINT:
            hDC = BeginPaint (hWnd, &ps);

            //--- 문자열 직접 사용
            TextOut (hDC, 0, 0, L"Hello World", strlen("Hello World"));

            //--- 변수 사용
            num = wcslen (str);
            TextOut (hDC, 0, 100, str, num);

            EndPaint (hWnd, &ps);
            break;

        case WM_DESTROY:
            PostQuitMessage ( 0 );
            break;
    }
    return DefWindowProc (hWnd, iMsg, wParam, lParam);
}
```



박스 영역에 텍스트 출력 함수: DrawText ()

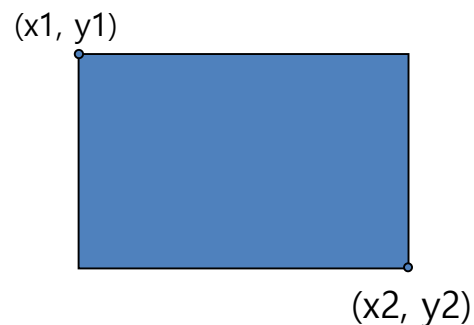
- 박스 영역에 텍스트 출력 함수

```
int DrawText ( HDC hDC, LPCSTR lpString, int nLength, LPRECT lpRect, UINT Flags);
```

- HDC hDC: BeginPaint()나 GetDC()를 통해 얻어온 DC핸들
- LPCSTR lpString: 출력 문자열
- int nLength: 문자열 길이
- LPRECT lpRect: 문자열을 출력할 박스 영역 구조체의 주소 (RECT *)
- UINT Flags: 출력 방법

- RECT 구조체

```
typedef struct tagRECT {  
    LONG left;           //--- x1  
    LONG top;            //--- y1  
    LONG right;          //--- x2  
    LONG bottom;         //--- y2  
} RECT;
```

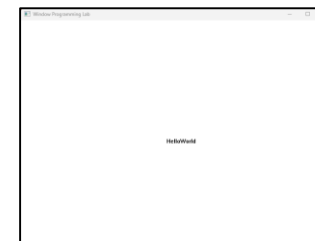


DrawText() 출력 방법

- **DrawText 함수의 Flag:**
 - DT_LEFT: 박스 영역 내에서 왼쪽 정렬
 - DT_CENTER: 박스 영역 내에서 가운데 정렬
 - DT_RIGHT: 박스 영역 내에서 오른쪽 정렬
 - DT_TOP: 박스 영역의 상하에서 위쪽에 출력
 - DT_BOTTOM: 박스 영역의 상하에서 아래쪽에 출력
 - DT_VCENTER: 박스 영역의 상하에서 가운데 출력 (DT_SINGLELINE 과 함께 사용)
 - DT_SINGLELINE: 박스 영역 안에 한 줄로 출력 (세로에 해당하는 플래그 사용 시 함께 사용)
 - DT_WORDBREAK: 단어가 박스 영역의 오른쪽 끝에 닿으면 단어 단위로 줄바꿈
 - DT_EDITCONTROL: DT_WORDBREAK에 OR해주면 영문,한글,숫자 모두 글자 단위로 개행

- **사용 예) 1개 이상의 설정을 하는 경우: 사각형 영역의 가운데 출력**

```
WCHAR szText[] = L"Hello world";  
RECT rect = {0, 0, 800, 600};  
int num;  
num = wcslen(szText);  
DrawText (hDC, szText, num, &rect, DT_CENTER | DT_VCENTER | DT_SINGLELINE);
```



- **사용 예) 긴 문자열을 사각 영역에 맞추어 출력하는 경우**

```
WCHAR szText[] = L"HelloWorld HelloWorld HelloWorld HelloWorld HelloWorld";  
RECT rect = {0, 0, 50, 600};  
int num;  
num = wcslen(szText);  
DrawText (hDC, szText, num, &rect, DT_WORDBREAK | DT_EDITCONTROL);
```



DrawText() 함수 이용하기

- DrawText 함수를 이용하여 HelloWorld 출력

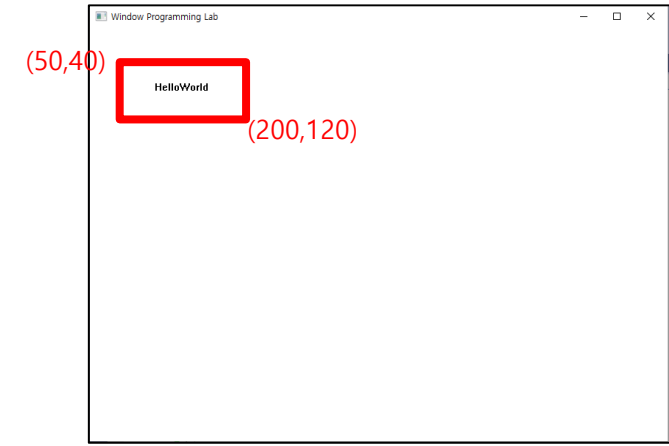
```
LRESULT CALLBACK WndProc (HWND hWnd, UINT iMsg, WPARAM wParam, LPARAM lParam)
{
    HDC hDC;
    PAINTSTRUCT ps;
    RECT rect;

    switch (iMsg)
    {
        case WM_PAINT:
            hDC = BeginPaint (hWnd, &ps) ;

            rect.left = 50;           //--- 사각형 정의
            rect.top = 40;
            rect.right = 200;
            rect.bottom = 120;

            DrawText (hDC, L"HelloWorld", 10, &rect, DT_SINGLELINE | DT_CENTER | DT_VCENTER);
            EndPaint (hWnd, &ps) ;
            break;

        case WM_DESTROY:
            PostQuitMessage ( 0 );
            break;
    }
    return DefWindowProc (hWnd, iMsg, wParam, lParam);
}
```



//--- 한 라인, 수직/수평 중앙

<문자열 만들기>

• 문자열 만들기

- int **wsprintf** (LPTSTR lpOut, LPCTSTR lpFmt...);
 - 서식화된 문자열을 버퍼에 저장한다.
- int **swprintf** (wchar_t *buffer, size_t count, const wchar_t *format, ...);
 - WCHAR 형의 서식화된 문자열을 버퍼에 저장한다.
- 사용 예)

```
LRESULT CALLBACK WndProc (HWND hWnd, UINT iMsg, WPARAM wParam, LPARAM lParam)
{
    HDC hDC;
    PAINTSTRUCT ps;
    TCHAR lpOut[100];
    int x=0, y=0;

    switch (iMsg)
    {
        case WM_PAINT:
            hDC = BeginPaint (hWnd, &ps);
            wsprintf (lpOut, L"%d * %d = %d", x, y, x * y);
            TextOut (hDC, x, y, lpOut, lstrlen (lpOut));
            EndPaint (hWnd, &ps);
            break;

        case WM_DESTROY:
            PostQuitMessage ( 0 );
            break;
    }
    return DefWindowProc (hWnd, iMsg, wParam, lParam);
}
```

문자 출력 시 배경색, 전경색 모드 지정

- 문자 색 및 배경색 변경 해주는 함수

COLORREF SetBkColor (HDC hDC, COLORREF crColor);

- hDC: 디바이스 컨텍스트 핸들
- crColor: 변경 할 배경색

COLORREF SetTextColor (HDC hDC, COLORREF crColor);

- hDC : 디바이스 컨텍스트 핸들
- crColor: 변경 할 문자색

- 사용 예) 앞 페이지의 예에서

case **WM_PAINT**:

hDC = BeginPaint (hWnd, &ps) ;

rect.left = 50; //--- 사각형 정의

rect.top = 40;

rect.right = 200;

rect.bottom = 120;

SetTextColor (hDC, RGB (255, 0, 0));

//--- 문자 출력 이전에 문자 색상 설정, 설정 이후의 문자는 빨간색으로 출력된다.

DrawText (hDC, L"HelloWorld", 10, &rect, DT_SINGLELINE | DT_CENTER | DT_VCENTER);

//--- 한 라인, 수직/수평 중앙

EndPaint (hWnd, &ps) ;

break;

문자 출력 시 배경색, 전경색 모드 지정

- 윈도우의 색 지정: RGB(Red, Green, Blue) 함수 사용

COLORREF RGB (BYTE R, BYTE G, BYTE B);

- R, G, B: 빛의 3원색으로 0 ~ 255 사이의 정수값
- 0 ~ 16777215 사이의 색상값 설정

- COLORREF: RGB 색상을 지정하는데 사용되는 DWORD 형식의 값
 - 16진수 형태
 - 0x00bbggrr의 값으로 저장됨
 - r, g, b 값을 얻으려면 각각 GetRValue, GetGValue, GetBValue 함수를 사용
- RGB 매크로 함수는 빨강, 초록, 파랑 색의 값을 인자로 받아 COLORREF 타입의 색상값으로 만든다.

```
COLORREF RGB {  
    BYTE byRed,           // 색상 중 빨간 색  
    BYTE byGreen,         // 색상 중 초록 색  
    BYTE byBlue           // 색상 중 파란 색  
}
```

- 사용 예)

```
COLORREF text_color ;  
text_color = RGB (255, 0, 0);
```

```
SetTextColor (hDC, text_color );
```

```
DrawText (hDC, "HelloWorld", 10, &rect, DT_SINGLELINE | DT_CENTER | DT_VCENTER);
```